

Relatório de Acompanhamento - Sprint 2: Projeto FoodGuard

1. Implementação do Frontend

Nesta segunda sprint, o foco do lado do usuário foi estabelecer as interfaces principais de interação do usuário com a inteligência artificial e estruturar as portas de entrada da aplicação.

- **Interface do Chat e Navegação:** Implementação da tela de conversa principal com a LLM, oferecendo suporte contínuo para o envio de texto. O histórico de conversas agora é mantido visível ao usuário, e foi adicionado um mecanismo de navegação que possibilita iniciar novas conversas de forma limpa.
- **Upload de Mídias:** A funcionalidade de envio de imagens foi desenvolvida e acoplada de forma integrada ao campo de envio do prompt.
- **Autenticação:** Foi dado o pontapé inicial na estruturação das telas de Login e Cadastro, preparando o terreno para a implementação das lógicas de segurança.
- **Design & UI:** O layout e a identidade visual da Landing Page do projeto foram definidos.

2. Desenvolvimento do Backend

No backend, a sprint concentrou-se em erguer a infraestrutura de dados, expor a API REST dos domínios de conversa e, principalmente, construir o núcleo de inteligência artificial que cruza o perfil de saúde do usuário com a composição dos alimentos.

- **Arquitetura e Stack:** Estruturação do servidor em Django 4.2 + Django REST Framework, organizado em camadas bem definidas — modelos e regras de negócio em `core/`, lógica de IA em `core/services/`, e a camada de API segmentada em serializers, views, urls e exceptions por domínio.
- **Conexão com Banco de Dados:** Integração com PostgreSQL 16 via `psycopg`, com credenciais parametrizadas por variáveis de ambiente (`django-environ`) e transações atômicas por requisição (`ATOMIC_REQUESTS`). Foram modeladas as entidades centrais do produto — Chat, Message e formulário de saúde — sobre um `BaseModel` abstrato (UUID como chave primária, flag `is_active` e timestamps automáticos), com suas respectivas migrations.
- **Criação das Rotas (API REST):** Implementação dos endpoints dos domínios de conversa e perfil:
 - **Chats:** listagem das conversas do usuário (mais recentes primeiro) e exclusão (com remoção em cascata das mensagens).

- **Mensagens:** envio de mensagem (que dispara o pipeline de IA) e listagem paginada do histórico de um chat.
 - **Formulário de saúde:** criação, consulta e atualização do perfil clínico-alimentar.
 - Documentação automática da API via OpenAPI/Swagger (drf-spectacular).
- **Integração com a LLM (Google Gemini):** Encapsulamento do modelo gemini-2.5-flash em um serviço dedicado (GeminiClient). O fluxo de envio de mensagem valida o remetente, recupera ou cria o chat, persiste a mensagem do usuário, monta o contexto (formulário de saúde formatado + dados do produto vindos do OpenFoodFacts) e persiste a resposta da IA. Por privacidade (RQ002), dados pessoais como nome, e-mail e data de nascimento são deliberadamente excluídos antes do envio à LLM.
- **Uso do DSPy (Engenharia de Prompt Programática):** Em vez de prompts em texto solto, a interação com a LLM foi construída com DSPy, garantindo saída estruturada e confiável:
 - **Signature tipada (AssessFoodSafety):** define formalmente as entradas (formulário de saúde do usuário, ingredientes/aditivos/alérgenos do produto e a query do usuário) e a saída esperada.
 - **Saída estruturada via Pydantic (FoodSafetyAssessment):** a resposta é validada em um esquema fixo — verdict (SAFE/DANGEROUS), explanation (explicação técnica) e recommends_doctor (flag de encaminhamento médico).
 - **ChainOfThought + JSONAdapter:** raciocínio passo a passo do modelo com serialização confiável em JSON, eliminando parsing frágil de texto livre.
 - **Regras de operação embutidas na signature:** caráter estritamente informativo, proibição de prescrição médica, recomendação médica obrigatória diante de qualquer sintoma relatado e postura conservadora anti-alucinação (na dúvida, classifica como DANGEROUS).
- **Regras de Negócio:** implementação das regras do domínio — um formulário de saúde por usuário, invalidação automática de chats antigos quando ao formulário de saúde é atualizada, bloqueio de mensagens em chats inativos e um conjunto de exceções customizadas com códigos HTTP apropriados.

3. Integração Backend e Frontend

- A conexão síncrona entre o Frontend e o Backend foi estabelecida com sucesso no escopo atual. A integração garante que a interface desenvolvida consiga se comunicar adequadamente com os serviços da API da aplicação.

4. Garantia de Qualidade e Engenharia de Prompt (Testes)

Foram realizados testes e definições essenciais para garantir o funcionamento correto do cruzamento de informações com a LLM.

- **Planejamento de Testes:** Criação de uma planilha matriz de casos de teste. O documento foi estruturado para cobrir possíveis variações de tipos de usuários, formatos de imagens enviados e diversidade de prompts.
- **Arquitetura de Dados:** Definição do formato padrão do prompt (carga/payload de envio) e também da estrutura de retorno que a resposta da LLM deve adotar para ser exibida corretamente ao usuário.